1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA
DEPARTAMENTO DE ENGENHARIA ELECTROTÉCNICA E DE COMPUTADORES

Robotics (02000537)

Trabalho Laboratorial 0 — Parte 1: Operar o UFACTORY Lite 6

Labwork 0 — Part 1: Operating the UFACTORY Lite 6

Jorge Batista · **João Quintas** · **Carlos Azevedo**

1.º Semestre / 1st Semester, 2026/2027
Grupos de 2 estudantes / Groups of 2 students · Semana / Week 2

[PT] — Versão Portuguesa

Onde é que isto se encaixa

O Trabalho Laboratorial 0 está dividido em duas partes. A **parte 1 (este documento, semana 2)** é o *nível básico*: conhecer a máquina, ligar-se a ela, trabalhar em segurança e comandá-la a partir das ferramentas que o próprio robô fornece — sem instalar nada no vosso computador além de um browser. A **parte 2 (semana 3)** cobre o ciclo de vida de desenvolvimento e prepara o nível *intermédio* (computador próprio, projeto Python, IDE) e o nível *avançado* (ROS 2 + Gazebo).

Avaliação. O LW0 vale 0.4 valores, entregue com o LW1 e o LW2 na **Entrega 1 (01/11/2026)**, e avaliado em 75% pela entrega e 25% pelo acompanhamento em aula no Checkpoint 1 (semana 8), mais uma defesa oral pontuada individualmente.

Objetivos

- Identificar as partes do Lite 6 e a estrutura da sua cadeia cinemática.
- Ligar um computador ao robô e chegar ao UFACTORY Studio.
- Compreender e aceitar a política de segurança do laboratório.
- Usar o Live Control para ativar o braço, movimentá-lo, ensinar poses e ler o seu estado.
- Executar os primeiros programas a partir da interface Python servida pelo robô.
- Obter o xArm Python SDK e correr os exemplos de `example/wrapper`.

1. A Máquina

1.1 Especificações

Propriedade	Valor
Graus de liberdade	6, todos rotativos
Alcance	440 mm
Carga útil	600 g, **incluindo a ferramenta**
Repetibilidade	±0.5 mm
Velocidade / aceleração máx. do TCP	500 mm/s / 50 000 mm/s^2
Velocidade / aceleração máx. das juntas	180°/s / 1145°/s^2
Massa do braço	7.2 kg; a caixa de controlo está integrada na base
Flange da ferramenta	DIN ISO 9409-1-A50/63 (M5)
Ferramenta nas bancadas	Lite 6 Gripper: dois dedos, 24 V, **curso de 16 mm**, 6 N, 284.6 g
Comunicação	Ethernet; UFACTORY Studio servido no porto **18333**

1.2 Partes

- **Base e caixa de controlo integrada.** Ao contrário dos modelos xArm maiores, o Lite 6 não tem armário separado: fonte de alimentação, controlador e porta de rede estão na base. A **etiqueta com o endereço IP** está na base — vão precisar dela no §2.
- **Seis elos e seis juntas**, numeradas de J1 (rotação da base) a J6 (rotação da ferramenta). Nem todas as juntas rodam livremente: J2 e J5, em particular, são restringidas, e todos os cálculos que escreverem este semestre têm de respeitar estas gamas.

Junta	J1	J2	J3	J4	J5	J6
Gama	±360°	±150°	−3.5° a +300°	±360°	±124°	±360°

- **Flange e ferramenta.** A garra é aparafusada à flange e é alimentada e comandada pelo conector de I/O da ferramenta. O seu curso de 16 mm é a razão pela qual as peças de jogo do Desafio Final têm um pino de 11 mm.
- **Botão de paragem de emergência**, ligado à caixa de controlo. Faz parte da célula, não é um acessório.
- **Indicadores de estado** na base: alimentação do robô, alimentação de 24 V, estado de movimento.

1.3 A cadeia cinemática, informalmente

Um manipulador é uma cadeia de corpos rígidos. O controlador conhece seis números — os ângulos das juntas — e a partir deles calcula onde está a ferramenta. Esse cálculo é a **cinemática direta**, e é o tema do LW1. Hoje vão ler ambas as grandezas (ângulos e pose) a partir do robô e observar que se movem em conjunto; a partir do LW1 passarão a calcular uma a partir da outra.

Why this matters in practice

Unidades e convenções, anotadas já para evitar um erro mais tarde. O Python SDK trabalha por omissão em **milímetros e graus**; o protocolo subjacente e o ROS 2 trabalham em **radianos**. Uma pose do TCP é reportada como $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}]$ no referencial da base — mas *qual* a convenção de rotação que esses três ângulos seguem não está no manual e não deve ser adivinhada. Determiná-la experimentalmente é uma tarefa do LW1.

2. Ligação

Cada robô difunde a sua **própria rede Wi-Fi, sem palavra-passe**. Vários portáteis podem estar ligados ao mesmo robô em simultâneo, mas o controlador executa um único fluxo de comandos de movimento de cada vez — coordenem-se dentro da bancada.

1. Liguem-se à rede Wi-Fi da bancada.
2. Leiam o **IP da caixa de controlo** na etiqueta da base do braço (`192.168.1.xxx`).
3. Coloquem o computador na mesma sub-rede: um endereço estático como `192.168.1.50/24`, com um **último octeto diferente em cada portátil da bancada** — combinem quem fica com qual.
4. Verifiquem com `ping <ip_do_robô>` antes de abrir mais alguma coisa. Se o ping falhar, nada mais funcionará e o problema é de rede, não do robô.
5. Abram `http://<ip_do_robô>:18333` num browser. O UFACTORY Studio é servido pela própria caixa de controlo; não há nada a instalar.

Bancada	SSID Wi-Fi	IP do controlador	URL do Studio
1	`Robot-__`	`192.168.1.___`	`http://<IP>:18333`
2	`Robot-__`	`192.168.1.___`	`http://<IP>:18333`

Why this matters in practice

Vejam primeiro o Studio. Sempre que algo não funcionar — um script que não faz nada, um movimento recusado — abram o Studio antes de abrirem o editor. Ele diz-vos de imediato se o braço está ativado, em que modo e estado está e que erro ficou registado. A maior parte do que parece um erro de software nesta unidade curricular é um braço que não está ativado.

3. Política de Segurança

Safety Gate – Simulation Before Hardware

1. A **paragem de emergência** está fisicamente ao alcance de um dos membros do grupo sempre que o braço está alimentado. Esse membro **não** é o que está a escrever código.
2. **Sem mãos dentro do envelope de trabalho** enquanto o braço está ativado.
3. **Nada corre no robô real sem ter corrido antes em simulação** e sido mostrado ao docente. A partir da parte 2 terão simuladores; nesta primeira sessão todo o movimento é feito à mão no Live Control ou é um dos exemplos fornecidos, corrido sob supervisão.
4. O **modo reduzido está ativo** em todas as sessões com hardware até à validação do Desafio Final: velocidade das juntas limitada a 30% (54°/s), velocidade do TCP limitada a 150 mm/s, e uma fronteira do TCP que envolve a bancada. O limite só é levantado com autorização do docente.
5. A **sensibilidade de colisão** nunca é reduzida abaixo do valor por omissão do laboratório e nunca é posta a 0.
6. A **carga do TCP** tem de estar corretamente definida antes de usar o modo manual (de ensino): com uma carga errada o braço não fica parado quando o largam.
7. Qualquer comportamento inesperado — movimento inexplicado, erro do controlador, erro de junta, "End module communication error" — é reportado ao docente **antes** de o braço ser reativado.
8. O robô é uma máquina de 7.2 kg capaz de mover a ferramenta a 500 mm/s. Tratem um braço alimentado como tratariam um motor em funcionamento.

4. UFACTORY Studio — Live Control

O Studio é a aplicação web do próprio robô. Os seus painéis: **Live Control** (movimentação, ensino, estado), **Blockly** (programação por blocos — nada do que é avaliado nesta unidade curricular é escrito em Blockly), **Python IDE** (§5), **Recording** e **Settings**.

4.1 A sequência de ativação

O braço não se move enquanto não acontecerem três coisas, por esta ordem, e que têm de ser repetidas após cada erro e após cada libertação da paragem de emergência:
1. limpar qualquer erro ou aviso registado;
2. **ativar o movimento** (os servos são energizados);
3. definir o **modo** (0 = controlo de posição) e depois o **estado** (0 = pronto).

No Studio isto é o botão **Enable Robot** mais a limpeza de erros; em Python é `clean_error()`, `motion_enable(True)`, `set_mode(0)`, `set_state(0)`. Reconhecer esta sequência nos dois sítios é a razão para a fazer nos dois sítios.

4.2 O que exercitar

- **Movimentação por juntas.** Movam cada junta individualmente e encontrem os extremos da sua gama. Comparem o que observam com a tabela do §1.2.
- **Movimentação cartesiana.** Movam a ferramenta em x, y, z e em roll, pitch, yaw. Reparem em que juntas se movem quando comandam uma direção puramente cartesiana — essa relação é o Jacobiano, e é o LW3.
- **Leitura de estado.** Identifiquem no ecrã: os seis ângulos das juntas, a pose do TCP, o modo e o estado atuais, a carga do TCP, a sensibilidade de colisão e a configuração de montagem.
- **Modo de ensino (manual).** Definam primeiro a carga do TCP. Passem depois ao modo manual e guiem o braço à mão. Sintam como o braço se sustenta contra a gravidade.
- **A garra.** Abram e fechem-na a partir do Studio. Meçam a abertura nos dois estados com um paquímetro, e a distância da face da flange à ponta dos dedos.
- **Erros de propósito.** Sob supervisão, encostem suavemente o braço à bancada até a deteção de colisão disparar. Observem o que o Studio reporta e pratiquem a recuperação: limpar, ativar, repor o estado. Saber como se apresenta um erro registado vale mais do que nunca ter visto nenhum.
- **Settings.** Localizem o modo reduzido, a fronteira do TCP, a sensibilidade de colisão e a carga do TCP. Não os alterem sem o docente.

5. A Interface Python Servida pelo Robô

O Studio inclui um **Python IDE** que corre scripts na própria caixa de controlo. É o caminho mais rápido para um primeiro programa: sem instalação, sem ambiente virtual, sem configuração de rede além do §2. A partir da parte 2 passarão para o vosso computador e para um editor a sério, e este painel ficará apenas para verificações rápidas.

Um programa mínimo:

```
from xarm.wrapper import XArmAPI

arm = XArmAPI('192.168.1.xxx', is_radian=False)    # mm e graus
arm.clean_warn(); arm.clean_error()
arm.motion_enable(enable=True)
arm.set_mode(0)                                    # controlo de posicao
arm.set_state(0)                                   # pronto

code, angles = arm.get_servo_angle()               # seis angulos de junta, graus
code, pose   = arm.get_position()                  # [x, y, z, roll, pitch, yaw]
print('code', code, 'angles', angles, 'pose', pose)

arm.set_servo_angle(angle=[0, 0, 0, 0, 0, 0], speed=20, wait=True)
arm.disconnect()
```

Why this matters in practice

Todas as chamadas devolvem primeiro um código de estado. Um código diferente de zero é um erro a tratar, não a ignorar. Um script que imprime uma pose enquanto descarta silenciosamente o código que dizia que o movimento foi recusado é um script que vos vai enganar durante uma hora. Escrevam a verificação desde o primeiro programa: `if code != 0: raise RuntimeError(...)`.

5.1 Primeiros exercícios

1. Leiam e imprimam os ângulos das juntas e a pose do TCP. Movam o braço à mão em modo manual, leiam de novo, e relacionem os dois números.
2. Leiam as gamas das juntas reportadas pelo controlador e comparem-nas com o §1.2.
3. Definam uma configuração `home` para a vossa célula e comandem o braço até ela a partir de três configurações iniciais diferentes, a velocidade 20. Leiam os ângulos de volta de cada vez e quantifiquem a dispersão.
4. Movam a ferramenta 50 mm em $+z$ com `set_position(...)` e depois de volta. Reparem na diferença entre um comando no espaço das juntas e um comando cartesiano.
5. Abram e fechem a garra a partir de Python (`open_lite6_gripper()`, `close_lite6_gripper()`, `stop_lite6_gripper()`).

6. O xArm Python SDK e os Seus Exemplos

A mesma API usada acima é distribuída num repositório que contém a documentação completa e um vasto conjunto de exemplos funcionais. Descarreguem-no agora — vão consultá-lo todo o semestre:

`https://github.com/xArm-Developer/xArm-Python-SDK`

```
git clone https://github.com/xArm-Developer/xArm-Python-SDK.git
```

Dois sítios interessam:

- `doc/api/xarm_api.md` — a referência da API. Mantenham-na aberta.
- `example/wrapper/` — exemplos executáveis, agrupados por família de robô.

6.1 Que exemplos se aplicam ao Lite 6

O Lite 6 é um braço de seis eixos, pelo que os exemplos em `example/wrapper/xarm6/` são a família aplicável; os de `common/` aplicam-se a todos os modelos. Leiam-nos por esta ordem:

Exemplo	O que mostra
`common/0002-get_property`	Leitura de estado: ângulos, pose, limites, códigos de erro
`xarm6/2001-move_joint`	Movimento no espaço das juntas com `set_servo_angle`
`common/1001-move_line`	Movimento cartesiano linear com `set_position`
`common/6001-set_reduced_mode`	Modo reduzido: limites de velocidade e fronteira do TCP
`common/3002/3003-record/playback_trajectory`	Gravar uma trajetória guiada à mão e reproduzi-la

Why this matters in practice

Os exemplos xarm6 não foram escritos para o Lite 6. Antes de correrem qualquer um deles, leiam-no e verifiquem três coisas: os **ângulos de junta** que comanda (foram escolhidos para as gamas do xArm6, não para as do Lite 6 — ver §1.2), as **velocidades**, e as **chamadas à garra**. A garra do xArm usa `set_gripper_position()`; a garra do Lite 6 é um dispositivo diferente e usa `open_lite6_gripper()` / `close_lite6_gripper()`. Correr num robô um exemplo que não foi lido é exatamente o hábito que esta unidade curricular existe para eliminar.

6.2 O que fazer com eles

Corram o `0002-get_property` e o `2001-move_joint` contra o robô da vossa bancada, em modo reduzido e sob supervisão, depois de os verificarem como acima. Para cada um, escrevam **três frases**: o que faz, que chamadas do SDK usa, e o que tiveram de alterar para o Lite 6.

7. Trabalho a Realizar — E0 Parte 1

1. **Ligação.** Estabeleçam a configuração de rede do §2 e cheguem ao Studio. Registem o SSID, o IP do robô e o IP estático do vosso portátil.
2. **Live Control.** Exercitem todos os pontos do §4.2. Registem a gama que observam em cada junta e as medições da garra.
3. **Python.** Completem os cinco exercícios do §5.1.
4. **Exemplos.** Completem o §6.2 para os dois exemplos indicados.
5. **Registo.** Escrevam `log_joints.py`: leiam os seis ângulos das juntas durante 30 s à cadência máxima que conseguirem sustentar, enquanto um colega guia o braço à mão em modo manual (carga definida primeiro). Tracem $\theta_{1..6}(t)$ e reportem a cadência de amostragem obtida.
6. **Sessão segura.** Escrevam `safe_session.py`: uma função que liga, limpa erros, ativa o movimento, define modo e estado, aplica o modo reduzido com os limites do §3 e uma fronteira do TCP para a vossa bancada, e devolve o objeto `XArmAPI`. Todos os scripts que escreverem no resto do semestre começam com esta chamada.
7. **Registo do grupo.** Nomes, números de estudante, turma PL.

Entregue com a Entrega 1 (01/11/2026)

- Uma secção de relatório com **máx. 3 páginas** cobrindo os pontos 1–5: a configuração de rede, as gamas de junta observadas face à especificação, as medições da garra, o resultado de cada exercício em Python, os resumos dos exemplos, o gráfico $\theta_{1..6}(t)$ e a cadência obtida, e o que observaram quando a deteção de colisão disparou.
- `safe_session.py` e `log_joints.py`.
- A **declaração de segurança assinada**, confirmando que ambos os membros leram e aceitaram o §3.

Reportar uma falha é útil; não reportar nada não é. Se algo não funcionou, escrevam o que viram e o que dizia a *primeira* mensagem de erro, copiada como texto e não fotografada.

Anexo — Problemas comuns nesta sessão

O `ping` falha
: Rede Wi-Fi errada, ou o portátil não está na sub-rede `192.168.1.x`, ou dois portáteis partilham o mesmo endereço.

O Studio não abre

O porto é o 18333 e o protocolo é `http`, não `https`.

O braço ignora todos os comandos

Não está ativado, ou há um erro registado, ou o estado não é 0. Façam a sequência do §4.1 no Studio e observem o que muda.

O braço cede ou cai em modo manual

A carga do TCP está errada. Definam-na e voltem a entrar em modo manual.

Um movimento é recusado com código diferente de zero

Quase sempre uma gama de junta, a fronteira do TCP do modo reduzido, ou uma pose sem solução. Imprimam o código e consultem-no no Studio.

"End module communication error"

O cabo da garra ou o conector de I/O da ferramenta não está bem encaixado. Reportem; não insistam às cegas.

Um script funciona uma vez e depois não faz nada

A execução anterior deixou o braço num estado diferente de zero, ou terminou sem fechar a ligação. Façam sempre `disconnect()` no fim.

[EN] — English Version

Where this fits

Labwork 0 is split in two. **Part 1 (this document, Week 2)** is the *basic level*: know the machine, connect to it, work safely, and drive it from the tools the robot itself provides — no installation on your computer beyond a browser. **Part 2 (Week 3)** covers the development lifecycle and sets up the *intermediate* level (your own computer, Python project, IDE) and the *advanced* level (ROS 2 + Gazebo).

Assessment. LW0 is worth 0.4 values, delivered with LW1 and LW2 in **Delivery 1 (01/11/2026)**, and assessed 75% on the delivery and 25% on class follow-up at Checkpoint 1 (Week 8), plus an individually scored oral defence.

Objectives

- Identify the parts of the Lite 6 and the structure of its kinematic chain.
- Connect a computer to the robot and reach UFACTORY Studio.
- Understand and accept the laboratory safety policy.
- Use Live Control to enable the arm, jog it, teach poses and read its state.
- Run first programs from the Python interface served by the robot.
- Obtain the xArm Python SDK and run its `example/wrapper` examples.

1. The Machine

1.1 Specifications

Property	Value
Degrees of freedom	6, all revolute
Reach	440 mm
Payload	600 g, **including the end tool**
Repeatability	±0.5 mm
Max. TCP speed / acceleration	500 mm/s / 50 000 mm/s^2
Max. joint speed / acceleration	180°/s / 1145°/s^2
Arm weight	7.2 kg; the control box is integrated in the base
Tool flange	DIN ISO 9409-1-A50/63 (M5)
End-effector on the benches	Lite 6 Gripper: two-finger, 24 V, **16 mm stroke**, 6 N, 284.6 g
Communication	Ethernet; UFACTORY Studio served on port **18333**

1.2 Parts

- **Base and integrated control box.** Unlike the larger xArm models, the Lite 6 has no separate cabinet: power supply, controller and network port are in the base. The **IP address label** is on the base — you will need it in §2.
- **Six links and six joints**, numbered J1 (base rotation) to J6 (tool rotation). The joints are not all free to turn: J2 and J5 in particular are restricted, and every computation you write this semester must respect these ranges.

Joint	J1	J2	J3	J4	J5	J6
Range	±360°	±150°	−3.5° to +300°	±360°	±124°	±360°

- **Tool flange and end-effector.** The gripper is bolted to the flange and powered and commanded through the tool I/O connector. Its 16 mm stroke is the reason the game pieces of the Final Challenge have an 11 mm peg.
- **Emergency stop button**, wired to the control box. It is part of the cell, not an accessory.
- **Status indicators** on the base: robot power, 24 V power, motion status.

1.3 The kinematic chain, informally

A manipulator is a chain of rigid bodies. The controller knows six numbers — the joint angles — and from them computes where the tool is. That computation is **forward kinematics**, and it is the subject of LW1. Today you will read both quantities (angles and pose) from the robot and observe that they move together; from LW1 onwards you will compute one from the other yourself.

Why this matters in practice

Units and conventions, noted now to avoid a bug later. The Python SDK works by default in **millimetres and degrees**; the underlying protocol and ROS 2 work in **radians**. A TCP pose is reported as $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}]$ in the base frame — but *which* rotation convention those three angles follow is not stated in the manual and is not to be guessed. Determining it experimentally is a task in LW1.

2. Connecting

Each robot broadcasts its **own Wi-Fi network, without password**. Several laptops may be connected to the same robot at once, but the controller executes one motion command stream at a time — coordinate within your bench.

1. Join the bench's Wi-Fi network.
2. Read the **control box IP** from the label on the base of the arm (`192.168.1.xxx`).
3. Put your computer on the same subnet: a static address such as `192.168.1.50/24`, with a **different last octet on every laptop at the bench** — agree on who takes which.
4. Verify with `ping <robot_ip>` before opening anything else. If the ping fails, nothing further will work and the problem is the network, not the robot.
5. Open `http://<robot_ip>:18333` in a browser. UFACTORY Studio is served by the control box itself; there is nothing to install.

Bench	Wi-Fi SSID	Controller IP	Studio URL
1	`Robot-__`	`192.168.1.___`	`http://<IP>:18333`
2	`Robot-__`	`192.168.1.___`	`http://<IP>:18333`

Why this matters in practice

Check Studio first. Whenever something does not work — a script that does nothing, a motion that is refused — open Studio before you open your editor. It tells you in one glance whether the arm is enabled, in what mode and state it is, and what error is latched. Most of what looks like a software bug in this course is an arm that is not enabled.

3. Safety Policy

⚠ Safety Gate – Simulation Before Hardware

1. The **emergency stop** is physically within reach of one group member at all times while the arm is powered. That member is **not** the one typing.
2. **No hands inside the working envelope** while the arm is enabled.
3. **Nothing runs on the real robot that has not run in simulation first** and been shown to the teacher. From Part 2 you will have simulators; in this first session every motion is either jogged by hand in Live Control or is one of the supplied examples, run under supervision.
4. **Reduced mode is on** for every hardware session until the Final Challenge demonstration: joint speed capped at 30% (54°/s), TCP speed capped at 150 mm/s, and a TCP boundary enclosing the bench. The cap is lifted only with teacher authorisation.
5. **Collision sensitivity** is never lowered below the laboratory default and never set to 0.
6. The **TCP payload** must be set correctly before using manual (teaching) mode: with a wrong payload the arm does not hold still when you let go of it.
7. Any unexpected behaviour — unexplained motion, controller error, joint error, "End module communication error" — is reported to the teacher **before** the arm is re-enabled.
8. The robot is a 7.2 kg machine that can move its tool at 500 mm/s. Treat a powered arm the way you would treat a running engine.

4. UFACTORY Studio — Live Control

Studio is the robot's own web application. Its panels: **Live Control** (jogging, teaching, state), **Blockly** (block programming — nothing graded in this course is written in Blockly), **Python IDE** (§5), **Recording**, and **Settings**.

4.1 The enable sequence

The arm will not move until three things have happened, in this order, and they must be repeated after every error and after every release of the emergency stop:
1. clear any latched error or warning;
2. **enable motion** (the servos are energised);
3. set the **mode** (0 = position control) and then the **state** (0 = ready).

In Studio this is the **Enable Robot** button plus the error-clearing prompt; in Python it is `clean_error()`, `motion_enable(True)`, `set_mode(0)`, `set_state(0)`. Recognising this sequence in both places is the point of doing it in both places.

4.2 What to exercise

- **Joint jogging.** Move each joint individually and find the ends of its range. Compare what you observe with the table in §1.2.
- **Cartesian jogging.** Move the tool along x, y, z and about roll, pitch, yaw. Watch which joints move when you command a purely Cartesian direction — that relationship is the Jacobian, and it is LW3.
- **State readout.** Identify on screen: the six joint angles, the TCP pose, the current mode and state, the TCP payload, the collision sensitivity and the mounting configuration.
- **Teaching (manual) mode.** Set the TCP payload first. Then switch to manual mode and guide the arm by hand. Feel how the arm holds itself against gravity.
- **The gripper.** Open and close it from Studio. Measure the opening in both states with a caliper, and the distance from the flange face to the fingertips.

- **Errors on purpose.** Under supervision, jog the arm gently into the bench so that collision detection triggers. Observe what Studio reports and practise the recovery: clear, enable, re-state. Knowing what a latched error looks like is worth more than never having seen one.
- **Settings.** Locate reduced mode, the TCP boundary, collision sensitivity and the TCP payload. Do not change them without the teacher.

5. The Python Interface Served by the Robot

Studio includes a **Python IDE** that runs scripts on the control box itself. It is the fastest way to a first program: no installation, no virtual environment, no network configuration beyond §2. From Part 2 you will move to your own computer and a real editor, and this panel will be used only for quick checks.

A minimal program:

```python
from xarm.wrapper import XArmAPI

arm = XArmAPI('192.168.1.xxx', is_radian=False)    # mm and degrees
arm.clean_warn(); arm.clean_error()
arm.motion_enable(enable=True)
arm.set_mode(0)                                    # position control
arm.set_state(0)                                   # ready

code, angles = arm.get_servo_angle()               # six joint angles, degrees
code, pose   = arm.get_position()                  # [x, y, z, roll, pitch, yaw]
print('code', code, 'angles', angles, 'pose', pose)

arm.set_servo_angle(angle=[0, 0, 0, 0, 0, 0], speed=20, wait=True)
arm.disconnect()
```

Why this matters in practice

Every call returns a status code first. A non-zero code is an error to handle, not to ignore. A script that prints a pose while silently discarding the code that said the motion was refused is a script that will mislead you for an hour. Write the check in from the first program: `if code != 0: raise RuntimeError(...)`.

5.1 First exercises

1. Read and print the joint angles and the TCP pose. Move the arm by hand in manual mode, read again, and relate the two numbers.
2. Read the joint ranges reported by the controller and compare them with §1.2.
3. Define a `home` configuration for your cell and command the arm to it from three different starting configurations at speed 20. Read back the joint angles each time and quantify the spread.
4. Move the tool 50 mm along $+z$ with `set_position(...)`, then back. Note the difference between a joint-space command and a Cartesian one.
5. Open and close the gripper from Python (`open_lite6_gripper()`, `close_lite6_gripper()`, `stop_lite6_gripper()`).

6. The xArm Python SDK and Its Examples

The same API used above is distributed as a repository containing the full documentation and a large set of worked examples. Download it now — you will read from it all semester:

`https://github.com/xArm-Developer/xArm-Python-SDK`

```
git clone https://github.com/xArm-Developer/xArm-Python-SDK.git
```

Two places matter:

- `doc/api/xarm_api.md` — the API reference. Keep it open.
- `example/wrapper/` — runnable examples, grouped by robot family.

6.1 Which examples apply to the Lite 6

The Lite 6 is a six-axis arm, so the examples under `example/wrapper/xarm6/` are the applicable family; those under `common/` apply to every model. Read them in this order:

Example	What it shows
`common/0002-get_property`	Reading state: angles, pose, limits, error codes
`xarm6/2001-move_joint`	Joint-space motion with `set_servo_angle`
`common/1001-move_line`	Cartesian linear motion with `set_position`
`common/6001-set_reduced_mode`	Reduced mode: speed caps and TCP boundary
`common/3002/3003-record/playback_trajectory`	Recording a hand-guided trajectory and replaying it

Why this matters in practice

The xArm6 examples are not written for the Lite 6. Before running any of them, read it and check three things: the **joint angles** it commands (they are chosen for the xArm6's ranges, not the Lite 6's — see §1.2), the **speeds**, and the **gripper calls**. The xArm gripper uses `set_gripper_position()`; the Lite 6 gripper is a different device and uses `open_lite6_gripper()` / `close_lite6_gripper()`. Running an unread example on a robot is exactly the habit this course exists to remove.

6.2 What to do with them

Run `0002-get_property` and `2001-move_joint` against your bench robot, in reduced mode and under supervision, after checking them as above. For each, write **three sentences**: what it does, which SDK calls it uses, and what you had to change for the Lite 6.

7. Work to be Performed — E0 Part 1

1. **Connection.** Establish the network configuration of §2 and reach Studio. Record SSID, robot IP and your laptop's static IP.
2. **Live Control.** Exercise every item of §4.2. Record the joint range you observe for each joint and the gripper measurements.
3. **Python.** Complete the five exercises of §5.1.
4. **Examples.** Complete §6.2 for the two named examples.
5. **Log.** Write `log_joints.py`: read the six joint angles for 30 s at the fastest rate you can sustain, while a colleague hand-guides the arm in manual mode (payload set first). Plot $\theta_{1..6}(t)$ and report the achieved sample rate.
6. **Safe session.** Write `safe_session.py`: a function that connects, clears errors, enables motion, sets mode and state, applies reduced mode with the caps of §3 and a TCP boundary for your bench, and returns the `XArmAPI` object. Every script you write for the rest of the semester starts with this call.
7. **Group registration.** Names, student numbers, PL group.

Submitted with Delivery 1 (01/11/2026)

- A report section of **max. 3 pages** covering items 1–5: the network configuration, the observed joint ranges against the specification, the gripper measurements, the outcome of each Python exercise, the example summaries, the $\theta_{1..6}(t)$ plot and the sample rate, and what you observed when collision detection triggered.
- `safe_session.py` and `log_joints.py`.
- The **signed safety declaration**, confirming that both members have read and accepted §3.

Reporting a failure is useful; reporting nothing is not. If something did not work, write what you saw and what the *first* error message said, copied as text rather than photographed.

Appendix — Common problems in this session

`ping` fails
Wrong Wi-Fi network, or your laptop is not on the `192.168.1.x` subnet, or two laptops share the same address.

Studio does not open
The port is 18333 and the protocol is `http`, not `https`.

The arm ignores every command
It is not enabled, or an error is latched, or the state is not 0. Do the sequence of §4.1 in Studio and watch what changes.

The arm drifts or falls in manual mode
The TCP payload is wrong. Set it, then re-enter manual mode.

A motion is refused with a non-zero code
Most often a joint range, the reduced-mode TCP boundary, or a pose that has no solution. Print the code and look it up in Studio.

"End module communication error"
The gripper cable or tool I/O connector is not seated. Report it; do not retry blindly.

A script works once and then does nothing
The previous run left the arm in a non-zero state, or disconnected without cleaning up. Always `disconnect()` at the end.